

JobMate

Plateforme de Services à la Demande

Rapport de Projet de Fin d'Études

Développement Full Stack — React 19 + Laravel 12

Marketplace de Services — Système de Réservation

Étudiant :	Youssef
Établissement :	Institut Bab Tizimi — Meknès, Maroc
Filière :	Développement Web Full Stack
Encadrant :	Dr. / M. [Nom de l'encadrant]
Année académique :	2025 — 2026

. Remerciements

Je tiens à exprimer ma sincère gratitude à mon encadrant pédagogique pour son accompagnement, ses conseils avisés et sa disponibilité tout au long de la réalisation de ce projet. Ses orientations ont été déterminantes dans la structuration et l'aboutissement de ce travail.

Je remercie également l'ensemble du corps enseignant de l'Institut Bab Tizimi de Meknès pour la qualité de la formation dispensée tout au long de mon parcours académique.

Enfin, je dédie ce travail à ma famille pour son soutien indéfectible, ainsi qu'à mes camarades de promotion pour les échanges enrichissants qui ont alimenté ma réflexion.

. Résumé

Ce rapport présente le développement de **JobMate**, une plateforme de mise en relation entre clients et prestataires de services à la demande. Le projet a été conçu et réalisé dans le cadre du projet de fin d'études de la filière Développement Web Full Stack à l'Institut Bab Tizimi, Meknès.

Aspect	Détail
Objectif	Créer une marketplace de services à la demande
Frontend	React 19, Redux Toolkit, React Router v7, Framer Motion
Backend	Laravel 12, Sanctum, Eloquent ORM, REST API
Base de données	SQLite (dev) / MySQL (production)
Authentification	Laravel Sanctum — tokens Bearer
Upload fichiers	Laravel Storage — multipart/form-data
Résultats	Application full-stack fonctionnelle avec rôles, réservations et paiements

. Table des Matières

- 1. Introduction Générale**
- 2. Analyse des Besoins**
- 3. Étude de l'Existant**
- 4. Conception UML**
 - 4.1 Diagramme de Cas d'Utilisation
 - 4.2 Diagramme de Classes
 - 4.3 Diagrammes de Séquence
 - 4.4 Diagramme d'État — Réservations
 - 4.5 Diagrammes d'Activité
- 5. Architecture du Système**
- 6. Base de Données**
- 7. Technologies Utilisées**
- 8. Réalisation**
- 9. Tests et Validation**
- 10. Conclusion Générale**
- 11. Annexes**

1. Introduction Générale

1.1 Contexte

La transformation numérique modifie profondément les pratiques de consommation de services. De plus en plus d'individus recherchent des prestataires qualifiés pour des besoins ponctuels (plomberie, électricité, bricolage, design d'intérieur, ménage) sans disposer d'outils fiables pour identifier, évaluer et réserver ces professionnels en ligne.

Dans ce contexte, les plateformes de mise en relation — comme Thumbtack, Angi ou Helpling — ont connu une croissance exponentielle. Le marché francophone reste cependant sous-exploité comparativement aux marchés anglophones.

1.2 Problématique

Les particuliers et entreprises font face à plusieurs problèmes concrets :

- Difficulté à identifier des prestataires fiables et vérifiés dans leur région.
- Absence de mécanisme de paiement sécurisé pour des services à domicile.
- Manque de visibilité pour les artisans et indépendants sur leur marché local.
- Processus de réservation fragmenté : appels téléphoniques, devis informels, absence de traçabilité.

1.3 Objectifs du Projet

Le projet JobMate vise à :

- Créer une marketplace biface mettant en relation clients (demandeurs) et prestataires (offreurs).
- Implémenter un système de réservation avec workflow complet : demande → acceptation → paiement → réalisation → avis.
- Sécuriser les échanges d'informations sensibles (coordonnées, paiement).
- Offrir un panneau d'administration pour la modération de la plateforme.
- Produire une application full-stack moderne, responsive et production-ready.

1.4 Solution Proposée

JobMate est une application web full-stack développée avec React 19 côté client et Laravel 12 côté serveur. Elle expose une API REST consommée par le frontend. L'authentification repose sur Laravel Sanctum (tokens Bearer). Les fichiers sont stockés via le système de fichiers Laravel (storage/public). La persistance des données est assurée par une base SQLite en développement et MySQL en production.

2. Analyse des Besoins

2.1 Besoins Fonctionnels

Module Authentification

- Inscription avec photo de profil (client ou prestataire).
- Connexion par email/mot de passe avec token Sanctum.
- Déconnexion avec révocation du token.
- Renouvellement de session (GET /api/auth/me).
- Rôles distincts : Client, Prestataire, Admin.

Module Services

- Création d'annonces par les prestataires (titre, catégorie, description, prix, localisation).
- Upload de galerie photos et certificats/diplômes.
- Modification et suppression de ses propres annonces.
- Recherche et filtrage par catégorie, ville, pays, prix maximum.
- Consultation du profil complet d'un prestataire.

Module Réservations

- Création d'une demande de réservation par un client.
- Acceptation ou refus par le prestataire.
- Paiement de la commission (25%) pour débloquent les coordonnées.
- Marquage de la prestation comme terminée.
- Affichage des coordonnées client uniquement après paiement.

Module Avis

- Soumission d'un avis (note 1–5 + commentaire) uniquement sur réservation terminée.
- Un seul avis par réservation (contrainte unique en base).
- Affichage des avis sur le profil du prestataire.

Module Administration

- Tableau de bord : statistiques globales (utilisateurs, services, revenus).
- Gestion des utilisateurs : liste, vérification prestataire, suppression.
- Gestion des services : liste, changement de statut, suppression.
- Monitoring des réservations avec filtres par statut.

2.2 Besoins Non Fonctionnels

- Performance : pagination côté serveur, images lazy-loaded, requêtes optimisées (Eager Loading).

- Sécurité : hachage bcrypt, validation FormRequest, politiques Laravel, tokens Sanctum.
- Responsive Design : interface fonctionnelle sur mobile, tablette et desktop.
- Maintenabilité : architecture Redux Toolkit, séparation API layer / store / UI.
- Fiabilité : gestion globale des erreurs (intercepteurs Axios, toasts).

2.3 Contraintes Techniques

- PHP $\geq$ 8.2 requis pour Laravel 12.
- Node.js $\geq$ 18 requis pour Vite et React 19.
- Extension SQLite ou MySQL obligatoire.
- Lien symbolique storage:link pour les uploads publics.

3. Étude de l'Existant

3.1 Solutions Existantes

Plateforme	Points Forts	Limites pour ce contexte
Thumbtack (USA)	Matching automatisé, évaluations détaillées	Non disponible en France, interface anglaise uniquement
Helpling (Europe)	Nettoyage spécialisé, paiement intégré	Niche unique (ménage), pas de multi-catégories
Angi (USA)	Large catalogue, leads qualifiés	Modèle payant pour prestataires, non adapté marché francophone
LeBonCoin Services	Audience massive en France	Pas de système de réservation, pas de paiement sécurisé, pas d'avis vérifiés
Malt (France)	Profils vérifiés, paiement escrow	Orienté freelance IT, pas services à domicile

3.2 Positionnement de JobMate

JobMate se positionne comme une solution francophone couvrant plusieurs catégories (plomberie, électricité, design, bricolage, menuiserie, nettoyage), intégrant un workflow de réservation complet avec paiement de commission, gestion des diplômes et un système d'administration. Son avantage différenciateur est la protection des coordonnées du client jusqu'au paiement de la commission par le prestataire.

4. Conception UML

4.1 Diagramme de Cas d'Utilisation

Le système JobMate comprend trois acteurs principaux : le **Client** (particulier cherchant un service), le **Prestataire** (professionnel proposant ses services) et l'**Administrateur** (modérateur de la plateforme). Chaque acteur interagit avec des cas d'utilisation distincts.

Acteurs et leurs cas d'utilisation

Client

- S'inscrire / Se connecter
- Parcourir les annonces
- Filtrer par catégorie / prix / localisation
- Consulter le profil d'un prestataire
- Envoyer une demande de réservation
- Consulter ses réservations
- Laisser un avis (réservation terminée)
- Mettre à jour son avatar

Prestataire

- S'inscrire / Se connecter
- Créer une annonce de service
- Uploader galerie photos et certificats
- Modifier / Supprimer une annonce
- Consulter les réservations reçues
- Accepter ou refuser une réservation
- Payer la commission pour débloquer les coordonnées
- Marquer une prestation terminée

Administrateur

- Se connecter (compte admin)
- Consulter le tableau de bord (statistiques)
- Gérer les utilisateurs (liste, vérification, suppression)
- Gérer les services (statut, suppression)
- Monitorer les réservations

Code PlantUML — Cas d'utilisation

```

@startuml JobMate_UseCase
left to right direction
skinparam actorStyle awesome
skinparam packageStyle rectangle
skinparam usecase {
    BackgroundColor #DBEAFE
    BorderColor #1E3A8A
}

actor "Client" as C #22c55e
actor "Prestataire" as P #3b82f6
actor "Administrateur" as A #ef4444

rectangle "JobMate – Plateforme de Services" {
    usecase "S'inscrire" as UC1
    usecase "Se connecter" as UC2
    usecase "Parcourir les services" as UC3
    usecase "Filtrer les annonces" as UC4
    usecase "Consulter profil prestataire" as UC5
    usecase "Créer une réservation" as UC6
    usecase "Laisser un avis" as UC7
    usecase "Créer une annonce" as UC8
    usecase "Upload galerie & certificats" as UC9
    usecase "Gérer ses annonces" as UC10
    usecase "Accepter/Refuser réservation" as UC11
    usecase "Payer commission" as UC12
    usecase "Marquer terminé" as UC13
    usecase "Dashboard admin" as UC14
    usecase "Gérer utilisateurs" as UC15
    usecase "Modérer services" as UC16
}

C --> UC1
C --> UC2
C --> UC3
C --> UC4
C --> UC5
C --> UC6
C --> UC7

P --> UC1
P --> UC2
P --> UC8
P --> UC9
P --> UC10
P --> UC11
P --> UC12
P --> UC13

A --> UC2
A --> UC14
A --> UC15
A --> UC16

UC4 ..> UC3 : <>
UC9 ..> UC8 : <>
UC7 ..> UC6 : <>
UC12 ..> UC11 : <>

```

@enduml

4.2 Diagramme de Classes

Le modèle de données de JobMate comprend cinq entités principales : **User**, **Service**, **Booking**, **Review** et **ServiceMedia**. Leurs relations sont déduites directement du code Eloquent des modèles Laravel.

Description des entités

Entité : User

Représente tout utilisateur du système. Le champ role discrimine les trois profils.

- id : bigInteger PK
- name : string
- email : string UNIQUE
- password : string (hashed)
- phone : string nullable
- avatar : string nullable (chemin storage)
- role : enum(client,provider,admin)
- is_verified : boolean
- created_at, updated_at : timestamp

Entité : Service

Annonce publiée par un prestataire. Liée aux médias via ServiceMedia.

- id : bigInteger PK
- provider_id : FK → users.id
- title : string
- category : string
- description : text nullable
- price : decimal(8,2)
- city : string nullable
- country : string nullable
- video_url : string nullable
- status : enum(active,pending,rejected)
- created_at, updated_at : timestamp

Entité : Booking

Demande de réservation d'un client pour un service. Cycle de vie complet géré par statuts.

- id : bigInteger PK
- service_id : FK → services.id
- client_id : FK → users.id
- provider_id : FK → users.id
- client_message : text nullable
- amount : decimal(10,2)
- status : enum(pending,accepted,declined,completed)
- payment_status : enum(unpaid,paid,refunded)
- created_at, updated_at : timestamp

Entité : Review

Avis laissé par un client sur une prestation terminée. Un seul avis par booking (UNIQUE).

- id : bigInteger PK
- booking_id : FK → bookings.id UNIQUE
- provider_id : FK → users.id
- client_id : FK → users.id
- rating : tinyInteger (1–5)
- comment : text nullable
- created_at, updated_at : timestamp

Entité : ServiceMedia

Fichiers associés à un service (galerie ou certificats).

- id : bigInteger PK
- service_id : FK → services.id
- path : string (chemin ou URL externe)
- type : enum(gallery,certificate)
- created_at, updated_at : timestamp

Code PlantUML — Diagramme de Classes

```
@startuml
    JobMate_Classes
    skinparam classBackgroundColor #DBEAFE
    skinparam classBorderColor #1E3A8A
    skinparam classHeaderBackgroundColor #1E3A8A
    skinparam classHeaderFontColor #FFFFFF

    class User {
        +id : bigint PK
        +name : string
    }
```

```
+email : string <>
+password : string
+phone : string
+avatar : string
+role : enum
+is_verified : boolean
--
+isAdmin() : bool
+isProvider() : bool
+isClient() : bool
+getAvatarUrlAttribute() : string
}

class Service {
  +id : bigint PK
  +provider_id : bigint FK
  +title : string
  +category : string
  +description : text
  +price : decimal
  +city : string
  +country : string
  +video_url : string
  +status : enum
  --
  +getAverageRatingAttribute() : float
  +getReviewsCountAttribute() : int
}

class Booking {
  +id : bigint PK
  +service_id : bigint FK
  +client_id : bigint FK
  +provider_id : bigint FK
  +client_message : text
  +amount : decimal
  +status : enum
  +payment_status : enum
}

class Review {
  +id : bigint PK
  +booking_id : bigint FK <>
  +provider_id : bigint FK
  +client_id : bigint FK
  +rating : tinyint
  +comment : text
}

class ServiceMedia {
  +id : bigint PK
  +service_id : bigint FK
  +path : string
  +type : enum
  --
  +getUrlAttribute() : string
}
```

```
User "1" --o{ "0..*" Service : "provider_id"
User "1" --o{ "0..*" Booking : "client (client_id)"
User "1" --o{ "0..*" Booking : "provider (provider_id)"
User "1" --o{ "0..*" Review : "reviewsReceived"
Service "1" --o{ "0..*" Booking : "bookings"
Service "1" --o{ "0..*" ServiceMedia : "media"
Booking "1" --o| "0..1" Review : "review"
@enduml
```

4.3 Diagrammes de Séquence

Inscription (Register)

```
@startuml Sequence_Register
actor Client as C
participant "React Frontend" as FE
participant "authSlice.js" as STORE
participant "authApi.js" as API
participant "Laravel AuthController" as CTRL
database "SQLite/MySQL" as DB

C -> FE: Remplit le formulaire\n(nom, email, mot de passe, rôle, avatar)
FE -> STORE: dispatch(registerThunk(FormData))
STORE -> API: registerApi(formData)
API -> CTRL: POST /api/auth/register\nContent-Type: multipart/form-data
CTRL -> CTRL: Valide RegisterRequest\n(email unique, password min 8)
CTRL -> CTRL: Store avatar -> storage/avatars/
CTRL -> DB: User::create({...})
DB --> CTRL: User $user
CTRL -> DB: $user->createToken('jobmate')
DB --> CTRL: PlainTextToken
CTRL --> API: 201 { user, token }
API --> STORE: Retourne data
STORE -> STORE: saveSession(token, user)\nlocalStorage.setItem(...)
STORE --> FE: isLoggedIn = true
FE -> C: Redirection vers /
@enduml
```

Connexion (Login)

```
@startuml Sequence_Login
actor Utilisateur as U
participant "AuthModal.jsx" as MODAL
participant "authSlice.js" as STORE
participant "Laravel AuthController" as CTRL
database "DB" as DB

U -> MODAL: Saisit email + mot de passe
MODAL -> STORE: dispatch(loginThunk({email,password}))
STORE -> CTRL: POST /api/auth/login
CTRL -> DB: User::where('email',...)->first()
DB --> CTRL: $user
alt Mot de passe correct
    CTRL -> DB: createToken('jobmate')
    CTRL --> STORE: 200 { user, token }
    STORE -> STORE: saveSession(token, user)
    STORE --> MODAL: isLoggedIn=true
    MODAL -> MODAL: showToast + closeModal()
else Mot de passe incorrect
    CTRL --> STORE: 422 ValidationException
    STORE --> MODAL: error = 'Informations incorrectes'
    MODAL -> U: Affiche message d'erreur
end
@enduml
```

Création d'un Service

```
@startuml Sequence_CreateService
actor Prestataire as P
participant "ServiceFormModal.jsx" as FORM
participant "serviceSlice.js" as STORE
participant "ServiceController" as CTRL
database "DB" as DB

P -> FORM: Remplit le formulaire\n+ photos galerie + certificats
FORM -> STORE: dispatch(createService(formData))
STORE -> CTRL: POST /api/services\nAuthorization: Bearer {token}\nContent-Type:
multipart/form-data
CTRL -> CTRL: authorize('create', Service::class)\nServicePolicy::create() -> true
CTRL -> CTRL: Valide ServiceRequest
CTRL -> DB: $user->services()->create({...})
DB --> CTRL: $service
loop Pour chaque photo galerie
    CTRL -> CTRL: $file->store('services/gallery','public')
    CTRL -> DB: ServiceMedia::create({path, type:'gallery'})
end
loop Pour chaque certificat
    CTRL -> CTRL: $file->store('services/certificates','public')
    CTRL -> DB: ServiceMedia::create({path, type:'certificate'})
end
CTRL --> STORE: 201 { service transformé }
STORE -> STORE: services.unshift(payload)
STORE --> FORM: loading = false
FORM -> P: Toast 'Service créé !'
@enduml
```

Processus de Réservation

```
@startuml Sequence_Booking
actor Client as C
actor Prestataire as P
participant "ProfilePage.jsx" as PP
participant "bookingSlice.js" as STORE
participant "BookingController" as CTRL
database "DB" as DB

C -> PP: Consulte le profil + clique 'Envoyer une demande'
PP -> STORE: dispatch(requestBooking({service_id, client_message}))
STORE -> CTRL: POST /api/bookings\nAuthorization: Bearer {token_client}
CTRL -> CTRL: Vérifie client != provider
CTRL -> DB: Booking::create({status:'pending', payment_status:'unpaid'})
DB --> CTRL: $booking
CTRL --> STORE: 201 { booking }
STORE -> STORE: bookings.unshift(payload)
PP -> C: Toast + redirect vers /dashboard

note over P: Plus tard, le prestataire\nconsulte son dashboard
P -> CTRL: PUT /api/bookings/{id}/status\n{status: 'accepted'}
CTRL -> CTRL: BookingPolicy::updateStatus -> true
CTRL -> DB: $booking->update({status:'accepted'})
CTRL --> P: 200 { booking mis à jour }
@enduml
```

Paielement de la Commission

```
@startuml Sequence_Payment
actor Prestataire as P
participant "CheckoutPage.jsx" as CP
participant "bookingSlice.js" as STORE
participant "BookingController" as CTRL
database "DB" as DB

P -> CP: Consulte la page paiement /checkout/{id}
CP -> STORE: dispatch(fetchBookings()) si vide
STORE -> CTRL: GET /api/bookings
CTRL --> STORE: Liste des réservations
CP -> P: Affiche récapitulatif (25% du montant)
P -> CP: Remplit formulaire CB + soumet
CP -> STORE: dispatch(updateBookingStatus({\n bookingId, payment_status:'paid'\n}))
STORE -> CTRL: PUT /api/bookings/{id}/status\n{payment_status: 'paid'}
CTRL -> CTRL: Validation: required_without:status
CTRL -> DB: $booking->update({payment_status:'paid'})
CTRL -> DB: Charge client + provider relations
CTRL --> STORE: 200 { booking + provider_phone + client_phone }
STORE -> STORE: Met à jour booking dans state
CP -> P: Affiche coordonnées client\n+ animation succès +redirect
@enduml
```

Publication d'un Avis

```
@startuml Sequence_Review
actor Client as C
participant "Dashboard.jsx" as DASH
participant "reviewSlice.js" as STORE
participant "ReviewController" as CTRL
database "DB" as DB

note over C: Prestation dans l'état 'completed'
C -> DASH: Clique 'Laisser un avis'
DASH -> C: Affiche formulaire (étoiles + commentaire)
C -> DASH: Saisit note (1-5) + commentaire + soumet
DASH -> STORE: dispatch(addReview({booking_id, rating, comment}))
STORE -> CTRL: POST /api/reviews\nAuthorization: Bearer {token}
CTRL -> CTRL: Vérifie client == booking.client_id
CTRL -> CTRL: Vérifie booking.status == 'completed'
CTRL -> CTRL: Vérifie !booking.review (unique)
CTRL -> DB: Review::create({...})
DB --> CTRL: $review avec client chargé
CTRL --> STORE: 201 { review }
STORE -> STORE: byProvider[pid].unshift(review)
DASH -> C: Toast 'Avis publié !'
@enduml
```

4.4 Diagramme d'État — Réservations

Une réservation (Booking) dans JobMate suit un cycle de vie précis gouverné par deux champs indépendants : **status** (état métier) et **payment_status** (état financier). Ces deux états évoluent en parallèle selon les actions des acteurs.

```
@startuml BookingStateDiagram
skinparam state {
    BackgroundColor #DBEAFE
    BorderColor #1E3A8A
    StartColor #22c55e
    EndColor #ef4444
}

[*] --> pending : Client crée une réservation\n(POST /api/bookings)

pending --> accepted : Prestataire accepte\n(PUT ../status {status:'accepted'})
pending --> declined : Prestataire refuse\n(PUT ../status {status:'declined'})

accepted --> paid_contact : Prestataire paie commission\n(PUT ../status {payment_status:'paid'})

note right of paid_contact
    Les coordonnées du client
    sont révélées au prestataire
    (client_phone exposé dans la réponse)
end note

paid_contact --> completed : Prestataire marque terminé\n(PUT ../status {status:'completed'})

completed --> reviewed : Client laisse un avis\n(POST /api/reviews)

declined --> [*] : Fin (aucune suite)
reviewed --> [*] : Fin du cycle

state "payment_status" as PS {
    [*] --> unpaid
    unpaid --> paid
    paid --> refunded : Remboursement\n(admin uniquement)
}

@enduml
```

4.5 Diagrammes d'Activité

Authentification

```
@startuml Activity_Auth
start
:Utilisateur ouvre le modal;
:Saisie email + mot de passe;
if (Mode connexion ?) then (oui)
    :dispatch(loginThunk);
    :POST /api/auth/login;
    if (Credentials valides ?) then (oui)
        :saveSession(token, user);
        :isLoggedIn = true;
        :Fermer modal + Toast succès;
    else (non)
        :Afficher erreur;
        stop
    endif
endif
else (non – inscription)
    :Saisie nom, téléphone, rôle, avatar;
    :Construire FormData;
    :dispatch(registerThunk);
    :POST /api/auth/register;
    if (Email déjà utilisé ?) then (oui)
        :Retour 422;
        :Afficher erreur;
        stop
    else (non)
        :Créer user + stocker avatar;
        :Générer token Sanctum;
        :saveSession + isLoggedIn = true;
        :Redirection vers /;
    endif
endif
stop
@enduml
```

Réservation d'un Service

```
@startuml Activity_Booking
start
:Client consulte le profil prestataire;
if (Connecté ?) then (non)
    :openLogin() – Ouvrir modal;
    stop
else (oui)
endif
if (Rôle == client ?) then (non)
    :Toast erreur 'Prestataires exclus';
    stop
else (oui)
endif
:Saisie message optionnel;
:dispatch(requestBooking);
:POST /api/bookings;
if (Client == Prestataire ?) then (oui)
    :Retour 422;
    stop
else (non)
endif
:Créer Booking (status=pending);
:Retour 201;
:Toast + Redirect /dashboard;
|Prestataire|
:Consulte dashboard;
if (Accepter ?) then (oui)
    :PUT .../status {status:accepted};
    :Naviguer vers /checkout/{id};
    :Payer commission 25%;
    :Coordonnées client débloquées;
    :Réaliser la prestation;
    :PUT .../status {status:completed};
else (non – refus)
    :PUT .../status {status:declined};
endif
stop
@enduml
```

5. Architecture du Système

5.1 Vue d'Ensemble

JobMate suit une architecture **SPA + REST API** découplée. Le frontend React 19 communique exclusivement avec le backend Laravel 12 via des appels HTTP authentifiés par tokens Bearer. Les deux applications peuvent être déployées indépendamment.

Couche	Technologie	Rôle
Présentation	React 19 + Vite	SPA — interface utilisateur réactive
État global	Redux Toolkit	Gestion centralisée : auth, services, bookings, reviews
Routing client	React Router v7	Navigation SPA, guards de routes (ProtectedRoute, AdminRoute)
API Layer	Axios + intercepteurs	Appels HTTP, token Bearer, gestion erreurs globale
API Backend	Laravel 12 REST	Endpoints sécurisés, validation, réponses JSON
Auth	Laravel Sanctum	Tokens Bearer par utilisateur, révocation
Persistence	Eloquent ORM	Modèles, relations, migrations
Base de données	SQLite / MySQL	Stockage des données relationnelles
Fichiers	Laravel Storage	Avatars, galeries, certificats en storage/public

5.2 Architecture Frontend (React 19)

La structure du projet frontend suit une organisation par fonctionnalité :

```
Frontend/src/
├── api/                                # Couche API – fonctions Axios par module
│   ├── client.js                     # Instance Axios + intercepteurs token/401
│   ├── authApi.js                   # register, login, logout, me, uploadAvatar
│   ├── serviceApi.js                # getServices, getService, create, update, delete
│   ├── bookingApi.js                # createBooking, getBookings, updateStatus
│   ├── reviewApi.js                 # createReview, getProviderReviews
│   └── adminApi.js                   # dashboard, users, services, bookings CRUD
├── store/                             # Redux Toolkit – slices avec createAsyncThunk
│   ├── store.js                     # configureStore
│   ├── authSlice.js                 # login, register, logout, me, setAvatar
│   ├── serviceSlice.js              # fetchServices, fetchService, create, update, delete
│   ├── bookingSlice.js              # fetchBookings, requestBooking, updateStatus
│   └── reviewSlice.js               # fetchProviderReviews, addReview
├── router/
│   ├── guards.jsx                   # ProtectedRoute, AdminRoute, ProviderRoute
├── context/
│   ├── AuthModalContext.jsx         # openLogin, openRegister, closeModal
├── hooks/
│   ├── useToast.jsx                 # ToastProvider, useToast()
├── pages/                             # Pages principales + admin/
└── components/                       # Composants réutilisables
```

```

■■■ App.jsx                # BrowserRouter + Routes + Providers

```

5.3 Architecture Backend (Laravel 12)

```

Backend/
■■■ app/Http/
■   ■■■ Controllers/Api/
■   ■   ■■■ AuthController.php      # register, login, logout, me
■   ■   ■■■ ServiceController.php   # CRUD + upload médias
■   ■   ■■■ BookingController.php   # store, index, updateStatus
■   ■   ■■■ ReviewController.php    # store, providerReviews
■   ■   ■■■ AdminController.php     # dashboard, users, services, bookings
■   ■   ■■■ UploadController.php    # avatar upload
■   ■■■ Middleware/
■   ■   ■■■ RoleMiddleware.php       # Vérification du rôle utilisateur
■   ■■■ Requests/                   # FormRequest – validation centralisée
■■■ app/Models/                     # Eloquent ORM – relations, accessors, casts
■■■ app/Policies/                   # ServicePolicy, BookingPolicy
■■■ app/Providers/
■   ■■■ AppServiceProvider.php      # Enregistrement des politiques
■■■ database/
■   ■■■ migrations/                 # Schéma : users, services, bookings, reviews, media
■   ■■■ factories/                  # Factories pour seeders
■   ■■■ seeders/                    # DatabaseSeeder – données de démo
■■■ routes/
■   ■■■ api.php                     # Routes API publiques, protégées, admin
■■■ bootstrap/
■   ■■■ app.php                     # Configuration routage, middleware aliases

```

6. Base de Données

6.1 Modèle Logique de Données (MLD)

Le schéma relationnel comprend cinq tables avec les clés étrangères suivantes :

```
users(id PK, name, email UNIQUE, password, phone, avatar, role, is_verified, timestamps)

services(id PK, provider_id FK->users.id CASCADE, title, category, description,
         price, city, country, video_url, status, timestamps)

bookings(id PK, service_id FK->services.id CASCADE,
         client_id FK->users.id CASCADE,
         provider_id FK->users.id CASCADE,
         client_message, amount, status, payment_status, timestamps)

reviews(id PK, booking_id FK->bookings.id CASCADE UNIQUE,
        provider_id FK->users.id CASCADE,
        client_id FK->users.id CASCADE,
        rating, comment, timestamps)

service_media(id PK, service_id FK->services.id CASCADE,
              path, type, timestamps)
```

6.2 Dictionnaire de Données

Table	Colonne	Type	Contraintes	Description
users	id	BIGINT	PK AUTO_INCREMENT	Identifiant unique
users	name	VARCHAR(100)	NOT NULL	Nom complet
users	email	VARCHAR(191)	NOT NULL UNIQUE	Adresse email
users	password	VARCHAR(255)	NOT NULL (hashed)	Mot de passe hashé bcrypt
users	phone	VARCHAR(20)	NULL	Numéro de téléphone
users	avatar	VARCHAR(255)	NULL	Chemin fichier ou URL
users	role	ENUM	DEFAULT 'client'	client / provider / admin
users	is_verified	BOOLEAN	DEFAULT false	Prestataire vérifié ?
services	id	BIGINT	PK	Identifiant service
services	provider_id	BIGINT	FK users	Prestataire propriétaire
services	title	VARCHAR(150)	NOT NULL	Titre de l'annonce
services	category	VARCHAR(60)	NOT NULL	Catégorie métier
services	price	DECIMAL(8,2)	NOT NULL	Prix horaire en euros
services	status	ENUM	DEFAULT 'active'	active/pending/rejected
bookings	id	BIGINT	PK	Identifiant réservation
bookings	status	ENUM	DEFAULT 'pending'	pending/accepted/declined/completed

bookings	payment_status	ENUM	DEFAULT 'unpaid'	unpaid/paid/refunded
bookings	amount	DECIMAL(10,2)	NOT NULL	Montant total en euros
reviews	booking_id	BIGINT	FK UNIQUE	Une review par booking
reviews	rating	TINYINT(1)	$1 \leq x \leq 5$	Note de 1 à 5
service_media	path	VARCHAR(255)	NOT NULL	Chemin ou URL externe
service_media	type	ENUM	NOT NULL	gallery / certificate

7. Technologies Utilisées

React 19	<i>Frontend</i>	Bibliothèque UI déclarative. Composants fonctionnels, hooks (useState, useEffect, useRef, useCallback). Rendu performant via Virtual DOM.
Redux Toolkit	<i>State Management</i>	Gestion d'état centralisée avec createSlice et createAsyncThunk. Séparation claire API → Thunk → State → UI.
React Router v7	<i>Navigation</i>	Routage SPA côté client. Routes imbriquées pour l'admin (/admin/*). Guards de routes personnalisés.
Framer Motion	<i>Animations</i>	Animations déclaratives : transitions de pages, animations de liste (AnimatePresence), effets hover.
Axios	<i>HTTP Client</i>	Instance configurée avec intercepteurs : ajout automatique du token Bearer, redirection globale sur 401.
Vite 5	<i>Build Tool</i>	Bundler ultra-rapide. HMR instantané en développement. Build optimisé pour production.
Laravel 12	<i>Backend</i>	Framework PHP MVC. Eloquent ORM, middleware, FormRequest, policies, Artisan CLI.
Sanctum	<i>Authentification</i>	Authentification API par tokens Bearer. Révocation des tokens. Stateless — adapté aux SPA.
Eloquent ORM	<i>Base de données</i>	ORM ActiveRecord. Relations hasMany, belongsTo, hasManyThrough. Accessors, casts, eager loading.
SQLite / MySQL	<i>SGBD</i>	SQLite pour le développement local (zéro configuration). MySQL en production pour la performance.
Laravel Storage	<i>Fichiers</i>	Gestion des uploads : avatars, galeries, certificats. Lien symbolique storage:link pour accès public.
CSS3 / Variables CSS	<i>Styles</i>	Variables CSS pour la cohérence du design system. Glassmorphism, animations, responsive design.

8. Réalisation

8.1 Fonctionnalités Principales Implémentées

Authentification Complète

- Inscription avec upload avatar (multipart/form-data)
- Connexion par email/mot de passe → token Bearer Sanctum
- Déconnexion avec révocation du token côté serveur
- Réhydratation de session au rechargement (GET /api/auth/me)
- Modal d'authentification accessible depuis toutes les pages
- Rôles : client, prestataire, admin — protégés côté serveur

Marketplace de Services

- Création d'annonces par les prestataires avec galerie photos et certificats
- Filtrage multi-critères : catégorie, recherche texte, ville, pays, prix max
- Pagination côté serveur (12 annonces par page)
- Page profil détaillée : galerie, certifications, avis, formulaire de réservation
- ServiceCard responsive avec avatar prestataire et fallback initiales

Système de Réservation

- Création de demande avec message optionnel
- Workflow complet : En attente → Acceptée → Payée → Terminée
- Refus possible à l'étape En attente
- Paiement de commission (25%) pour débloquer les coordonnées
- Protection des données de contact jusqu'au paiement
- Tableau de bord distinct pour clients et prestataires

Système d'Avis

- Soumission uniquement sur prestation terminée (validé côté serveur)
- Un avis par réservation (contrainte UNIQUE en base)
- Notes 1 à 5 étoiles + commentaire textuel
- Affichage sur profil avec calcul de moyenne dynamique
- Avatar du client affiché dans chaque avis

Panneau d'Administration

- Tableau de bord : total utilisateurs, prestataires, services, réservations, revenus
- Gestion des utilisateurs : liste paginée, vérification prestataire, suppression
- Gestion des services : changement de statut (actif/en attente/rejeté), suppression
- Monitoring des réservations avec filtres et pagination
- Interface admin avec sidebar et routes imbriquées

8.2 Sécurité Implémentée

- Hachage bcrypt automatique via `$casts = ['password' => 'hashed']` dans le modèle User.
- Validation FormRequest côté serveur pour toutes les entrées utilisateur.
- Politiques Laravel (ServicePolicy, BookingPolicy) pour l'autorisation granulaire.
- RoleMiddleware : vérification du rôle pour les routes protégées.
- Les coordonnées téléphoniques ne sont jamais exposées avant paiement validé.
- Le rôle 'admin' ne peut jamais être assigné via l'API d'inscription.
- Intercepteur Axios : redirection automatique vers / sur réponse 401.
- CORS configuré pour n'autoriser que l'origine du frontend.

8.3 Tableau des Endpoints API

Méthode	Endpoint	Auth	Rôle	Description
POST	/api/auth/register	Non	—	Inscription avec avatar
POST	/api/auth/login	Non	—	Connexion, retourne token
POST	/api/auth/logout	Oui	Tous	Révocation token
GET	/api/auth/me	Oui	Tous	Profil utilisateur courant
POST	/api/upload/avatar	Oui	Tous	Mise à jour avatar
GET	/api/services	Non	—	Liste paginée + filtres
GET	/api/services/{id}	Non	—	Détail service + médias
POST	/api/services	Oui	provider/admin	Créer service + médias
PUT	/api/services/{id}	Oui	provider/admin	Modifier service
DELETE	/api/services/{id}	Oui	provider/admin	Supprimer service
GET	/api/bookings	Oui	Tous	Réservations de l'utilisateur
POST	/api/bookings	Oui	client	Créer une réservation
PUT	/api/bookings/{id}/status	Oui	Tous	Mettre à jour statut
POST	/api/reviews	Oui	client	Publier un avis
GET	/api/providers/{id}/reviews	Non	—	Avis d'un prestataire
GET	/api/admin/dashboard	Oui	admin	Statistiques globales
GET	/api/admin/users	Oui	admin	Liste utilisateurs
DELETE	/api/admin/users/{id}	Oui	admin	Supprimer utilisateur
PUT	/api/admin/users/{id}/verify	Oui	admin	Toggle vérification
GET	/api/admin/services	Oui	admin	Liste services
DELETE	/api/admin/services/{id}	Oui	admin	Supprimer service
PUT	/api/admin/services/{id}/status	Oui	admin	Changer statut
GET	/api/admin/bookings	Oui	admin	Liste réservations

9. Tests et Validation

9.1 Tests Fonctionnels

Scénario	Action	Résultat Attendu	Status
Inscription	Soumettre formulaire avec email unique et mot de passe	Utilisateur créé en base, token retourné, session sauvegardée	■
Inscription email existant	Réutiliser un email existant	Retour 422 + message 'email déjà utilisé'	■
Connexion valide	Email + mot de passe corrects	Token Bearer retourné, utilisateur connecté	■
Connexion invalide	Mauvais mot de passe	Retour 422 + message 'Informations incorrectes'	■
Création service	Prestataire crée une annonce avec service	Service créé, médias stockés, réponse 201	■
Création service non prestataire	Client tente de créer un service	Retour 403 Forbidden (RoleMiddleware)	■
Réservation	Client réserve un service	Booking créé status=pending, retour 201	■
Auto-réservation	Prestataire réserve son propre service	Retour 422 'ne peut pas réserver son propre service'	■
Paieement commission	Prestataire paie {payment_status: 'paid'}	Booking mis à jour, coordonnées exposées dans la réponse	■
Avis sur booking terminé	Client poste avis après prestation	Review créée, retour 201	■
Double avis	Client tente second avis même booking	Retour 422 'avis déjà laissé'	■
Accès admin non-admin	Client accède /api/admin/dashboard	Retour 403 Forbidden	■

10. Conclusion Générale

10.1 Résultats Obtenus

Le projet JobMate constitue une réalisation complète d'une application web full-stack de marketplace de services. Les objectifs fixés en début de projet ont été intégralement atteints :

- Application full-stack fonctionnelle avec frontend React 19 et backend Laravel 12.
- Système d'authentification sécurisé par tokens Sanctum avec trois niveaux de rôles.
- Workflow de réservation complet de la demande jusqu'à la publication d'avis.
- Panneau d'administration pour la modération de la plateforme.
- Système d'upload de fichiers (avatars, galeries, certificats).
- API REST documentée avec 23 endpoints sécurisés.
- Interface responsive, animée et production-ready.

10.2 Difficultés Rencontrées

- Synchronisation de l'état Redux avec les mises à jour backend — résolu par des thunks asynchrones structurés.
- Gestion des permissions multi-niveaux — résolu par le RoleMiddleware et les Policies Laravel.
- Protection des données sensibles (téléphone) — résolu par une logique backend conditionnelle au paiement.
- Upload multipart depuis React (remplacement du FileReader/base64) — résolu par FormData natif.
- Configuration du provider bootstrap/app.php (routes API non enregistrées) — résolu par l'ajout explicite de api:.

10.3 Perspectives d'Amélioration

- Intégration d'un système de messagerie en temps réel (WebSockets/Laravel Echo).
- Paiement réel via Stripe ou PayPal pour la commission prestataire.
- Système de notifications push (email + in-app) pour les changements de statut.
- Application mobile iOS/Android avec React Native partageant la même API.
- Géolocalisation pour filtrer les prestataires par distance.
- Tableau de bord analytique avancé pour les prestataires (revenus, taux de conversion).

11. Annexes

Annexe A — Structure Complète du Projet

```

JobMate/
├── Frontend/
│   ├── src/
│   │   ├── api/           # client.js, authApi.js, serviceApi.js, ...
│   │   ├── store/         # store.js, authSlice.js, serviceSlice.js, ...
│   │   ├── router/        # guards.jsx
│   │   ├── context/       # AuthModalContext.jsx
│   │   ├── hooks/         # useToast.jsx
│   │   ├── pages/         # HomePage, BrowsePage, ProfilePage, Dashboard, ...
│   │   │   ├── admin/     # AdminLayout, AdminDashboard, AdminUsers, ...
│   │   │   ├── components/ # ServiceCard, ServiceFormModal, ...
│   │   │   ├── common/    # Navbar, Footer
│   │   │   ├── auth/      # AuthModal
│   │   │   └── ui/        # Toast
│   │   ├── App.jsx
│   │   ├── main.jsx
│   │   └── index.css
│   ├── public/
│   ├── package.json
│   └── vite.config.js
├── Backend/
│   ├── app/
│   │   ├── Http/
│   │   │   ├── Controllers/Api/ # 6 controllers
│   │   │   ├── Middleware/      # RoleMiddleware.php
│   │   │   └── Requests/        # RegisterRequest, ServiceRequest, ...
│   │   ├── Models/             # User, Service, Booking, Review, ServiceMedia
│   │   ├── Policies/           # ServicePolicy, BookingPolicy
│   │   ├── Providers/          # AppServiceProvider
│   │   ├── database/
│   │   │   ├── migrations/     # 5 migrations
│   │   │   ├── factories/      # 4 factories
│   │   │   └── seeders/        # DatabaseSeeder (53 users, 21 services, 50 bookings,
80 reviews)
│   │   ├── routes/
│   │   │   ├── api.php
│   │   │   ├── bootstrap/app.php
│   │   │   ├── .env
│   │   │   └── composer.json

```

Annexe B — Commandes de Démarrage

```

# Backend Laravel
cd Backend
cp .env.example .env
php artisan key:generate
php artisan migrate:fresh --seed
php artisan storage:link

```

```

php artisan serve
# -> http://localhost:8000

# Frontend React
cd Frontend
npm install
echo 'VITE_API_URL=http://localhost:8000/api' > .env.local
npm run dev
# -> http://localhost:5173

# Comptes de démonstration
Admin      : admin@jobmate.fr / admin123
Provider   : marc@example.com / password
Client     : client@example.com / password

```

Annexe C — Extrait de Code : authSlice.js (thunk login)

```

export const loginThunk = createAsyncThunk(
  'auth/login',
  async (credentials, { rejectWithValue }) => {
    try {
      const { data } = await loginApi(credentials);
      localStorage.setItem('jobmate_token', data.token);
      localStorage.setItem('jobmate_user', JSON.stringify(data.user));
      return data.user;
    } catch (err) {
      const msg = err.response?.data?.message
        || Object.values(err.response?.data?.errors ?? {})[0]?.[0]
        || 'Connexion échouée.';
      return rejectWithValue(msg);
    }
  }
);

```

Annexe D — Extrait de Code : BookingController (paiement)

```

public function updateStatus(Request $request, Booking $booking): JsonResponse
{
    $this->authorize('updateStatus', $booking);

    $request->validate([
        'status' => [
            'sometimes', 'required_without:payment_status',
            'in:accepted,declined,completed',
        ],
        'payment_status' => [
            'sometimes', 'required_without:status',
            'in:unpaid,paid,refunded',
        ],
    ]);

    $update = [];
    if ($request->has('status')) $update['status'] = $request->status;
    if ($request->has('payment_status')) $update['payment_status'] =
    $request->payment_status;
}

```

```
$booking->update($update);

$data = $this->transform($booking->load('service', 'client', 'provider'));

// Coordonnées exposées uniquement après paiement confirmé
if ($booking->payment_status === 'paid') {
    $data['provider_phone'] = $booking->provider?->phone;
    $data['client_phone']   = $booking->client?->phone;
}

return response()->json($data);
}
```